

1. Funcionamiento general:

El sistema de valoraciones permite a los alumnos puntuar a un profesor una vez por clase completada. La valoración consta de una puntuación numérica (1-5 estrellas) y un comentario opcional. Las reseñas son públicas y visibles en la ficha del profesor junto con la media global y la distribución de puntuaciones.

2. Tablas de base de datos utilizadas:

La tabla reseñas ya existía en el esquema original de ISIMatch. No fue necesario crear ninguna tabla nueva.

Reseñas: Almacena cada valoración individual:

Columna	Tipo	Descripción
id	INT PK AUTO_INCREMENT	Identificador único de la reseña
clase_id	INT FK → clases.id	Clase que se está valorando
profesor_id	INT FK → usuarios.id	Profesor valorado
alumno_id	INT FK → usuarios.id	Alumno que realiza la valoración
puntuacion	INT CHECK (1-5)	Nota numérica de 1 a 5
comentario	TEXT NULL	Texto libre opcional (máx. 500 caracteres)
fecha_reseña	TIMESTAMP DEFAULT NOW()	Fecha y hora del envío

La constraint CHECK (puntuacion BETWEEN 1 AND 5) actúa como validación a nivel de base de datos.

Profesores_detalle: Almacena el resumen agregado

Columna	Tipo	Uso en valoraciones
usuario_id	INT PK FK	Identifica al profesor
valoracion_media	DECIMAL(3,2)	Media calculada por PHP tras cada nueva reseña
total_reseñas	INT	Contador total de reseñas recibidas

Estas columnas ya existían en el esquema. Estas columnas se actualizan mediante las funciones SQL AVG() y COUNT() cada vez que se registra una nueva valoración.

Clases: Control de estados

Columna relevante	Valor controlado
Estado	Solo las clases con estado = 'aceptada' y cuya fecha_hora ya ha pasado (NOW()) son valorables.
Alumno_id	Se verifica que coincida con el alumno en sesión.
Profesor_id	Se verifica que coincida con el profesor enviado en el POST.

3. Flujo técnico detallado

Dashboard del alumno (dashboard-alumno.php):

La consulta SQL se amplió para detectar con un solo viaje a la base de datos si cada clase ya fue valorada:

```
SELECT c.*, u.nombre AS profe_nombre, u.apellidos AS
profe_apellidos,
r.id AS reseña_id
FROM clases c
```

```
JOIN usuarios u ON c.profesor_id = u.id  
LEFT JOIN reseñas r ON r.clase_id = c.id AND r.alumno_id =  
c.alumno_id  
WHERE c.alumno_id = '{alumno_id}' AND c.estado = 'aceptada' AND  
fecha_hora < NOW()  
ORDER BY c.fecha_hora DESC  
LIMIT 5
```

Si reseña_id IS NOT NULL → se muestra el aviso de que ya está valorada.

Si reseña_id IS NULL → se muestra el botón 'valorar' enlazando a `valorar.php?id={clase_id}`

Formulario de valoración (valorar.php);

Al cargar la página el servidor ejecuta dos comprobaciones críticas:

1. La clase es válida y pertenece al alumno:

```
SELECT id FROM clases  
WHERE id = ? AND alumno_id = ? AND profesor_id = ? AND estado =  
'aceptada' AND fecha_hora < NOW()
```

2. No existe ya una reseña para esta clase:

```
SELECT id FROM resenas WHERE clase_id = ? AND alumno_id = ?
```

Si la clase ya fue valorada, se muestra una pantalla de confirmación en lugar del formulario, impidiendo cualquier intento de doble valoración.

El selector de estrellas está implementado en HTML/CSS puro (sin librerías externas) con el truco de `flex-direction: row-reverse` para que el hover de cada estrella ilumine también las anteriores.

Controlador de guardado (PHP/guardar_valoracion.php):

Aplica cuatro capas de validación en orden:

Capa	Qué comprueba?	Error devuelto
1	IDs numéricos y válidos (> 0)	¿Redirige al dashboard?
2	Puntuación entre 1 y 5	?error=puntuacion
3	La clase pertenece al alumno y está completada	?error=permisos o ?error=estado
4	No existe reseña previa para esa clase.	error=duplicada

Tras las validaciones, el guardado se hace en tres pasos:

1. Insertar la reseña:

```
INSERT INTO resenas (clase_id, profesor_id, alumno_id, puntuacion, comentario)
```

```
VALUES (?, ?, ?, ?, ?)
```

2. Recalcular el promedio:

```
SELECT COUNT(*) AS total, ROUND(AVG(puntuacion), 2) AS media  
FROM resenas WHERE profesor_id = ?
```

3. Actualizar la ficha del profesor

```
UPDATE profesores_detalle  
SET valoracion_media = ?, total_reseñas = ?  
WHERE usuario_id = ?
```

Se usa AVG() real en lugar de un cálculo incremental para evitar acumulación de errores de redondeo a lo largo del tiempo.

Ficha del profesor (ficha-profesor.php):

Se añadieron dos consultas adicionales:

1. Reseñas recientes (hasta 5):

```
SELECT r.puntuacion, r.comentario, r.fecha_resena, u.nombre  
FROM resenas r  
JOIN usuarios u ON r.alumno_id = u.id  
WHERE r.profesor_id = '{perfil_id}'  
ORDER BY r.fecha_resena DESC LIMIT 5
```

2. Distribución por puntuación (para las barras de porcentaje):

```
SELECT puntuacion, COUNT(*) AS cantidad  
FROM resenas  
WHERE profesor_id = '{perfil_id}'  
GROUP BY puntuacion ORDER BY puntuacion DESC
```

La distribución se convierte a porcentaje en PHP:

```
$pct = ($total_val > 0) ? round(($distribucion[$s] / $total_val) * 100)  
: 0;
```

4. Archivos del módulo:

Archivo	Tipo	Rol
---------	------	-----

VIEWS/valorar.php	Vista (nuevo)	Formulario de valoración con estrellas interactivas
PHP/guardar_valoracion.php	Controlador (nuevo)	Validación, INSERT y UPDATE de promedio
VIEWS/dashboard-alumno.php	Vista (modificado)	Botón "Valorar" / badge "Valorada" en historial
VIEWS/ficha-profesor.php	Vista (modificado)	Estadísticas de valoraciones, distribución y reseñas

5. Problemas encontrados y soluciones:

1. Las tablas ya existían, pero sin datos de prueba

Inicialmente se planteó utilizar un estado completada, pero durante las pruebas se comprobó que las clases permanecían con estado aceptada incluso después de haber finalizado.

Para resolverlo, se decidió considerar como realizada cualquier clase cuya fecha ya hubiera pasado:

AND c.fecha_hora < NOW()

De este modo, una clase es valorable cuando:

Pertenece al alumno, tiene estado aceptada y su fecha y hora ya han transcurrido.

2. Doble valoración por recarga del formulario

Si el alumno enviaba el formulario y luego recargaba la página de confirmación, el navegador podía reenviar el POST, generando una segunda reseña para la misma clase.

Backend: comprobación con SELECT antes del INSERT. Si ya existe una reseña para clase_id + alumno_id, se redirige con ?error=duplicada.

Frontend: al hacer submit, el botón se deshabilita inmediatamente con JavaScript (btnEnviar.disabled = true) para impedir un segundo clic.

Post-Redirect-Get: el controlador siempre termina con un header("Location: ..."), eliminando el POST del historial del navegador.

3. Las clases realizadas no aparecían en el panel del alumno

Durante las pruebas se observó que las clases cuya fecha ya había transcurrido seguían teniendo estado aceptada, por lo que no aparecían en el apartado "Clases realizadas" y tampoco podían ser valoradas.

La solución consistió en modificar las consultas SQL para considerar realizadas todas las clases cuya fecha fuera anterior a la fecha actual:

```
WHERE c.alumno_id = ?  
AND c.estado = 'aceptada'  
AND c.fecha_hora < NOW()
```

4. El botón «Valorar» abría una clase distinta

Durante las pruebas se detectó que el identificador enviado a valorar.php no coincidía con la clase mostrada en pantalla, provocando el error clase_invalida.

La causa era una consulta mal ejecutada en dashboard-alumno.php.

La solución consistió en ejecutar correctamente la consulta:

```
$res_recientes = $conn->query($sql_recientes);
```

garantizando que cada botón:

```
<a href="valorar.php?id=<?php echo $rec['id']; ?>">
```

enviara el identificador correcto de la clase correspondiente.